

吉林大学



第二章 导引

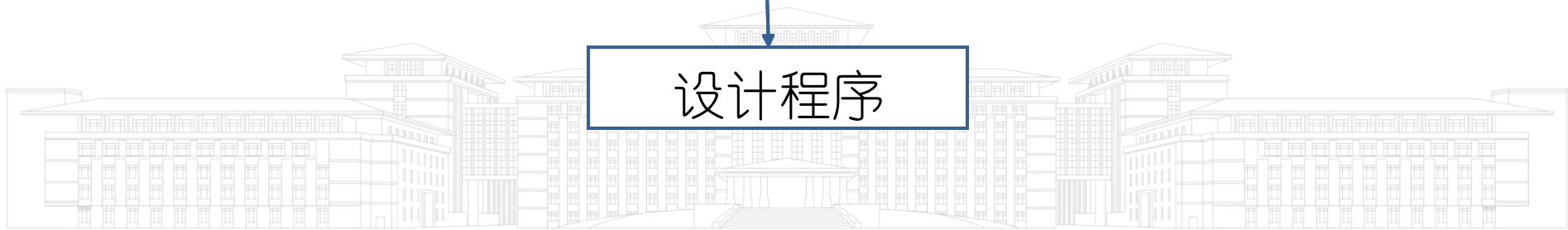
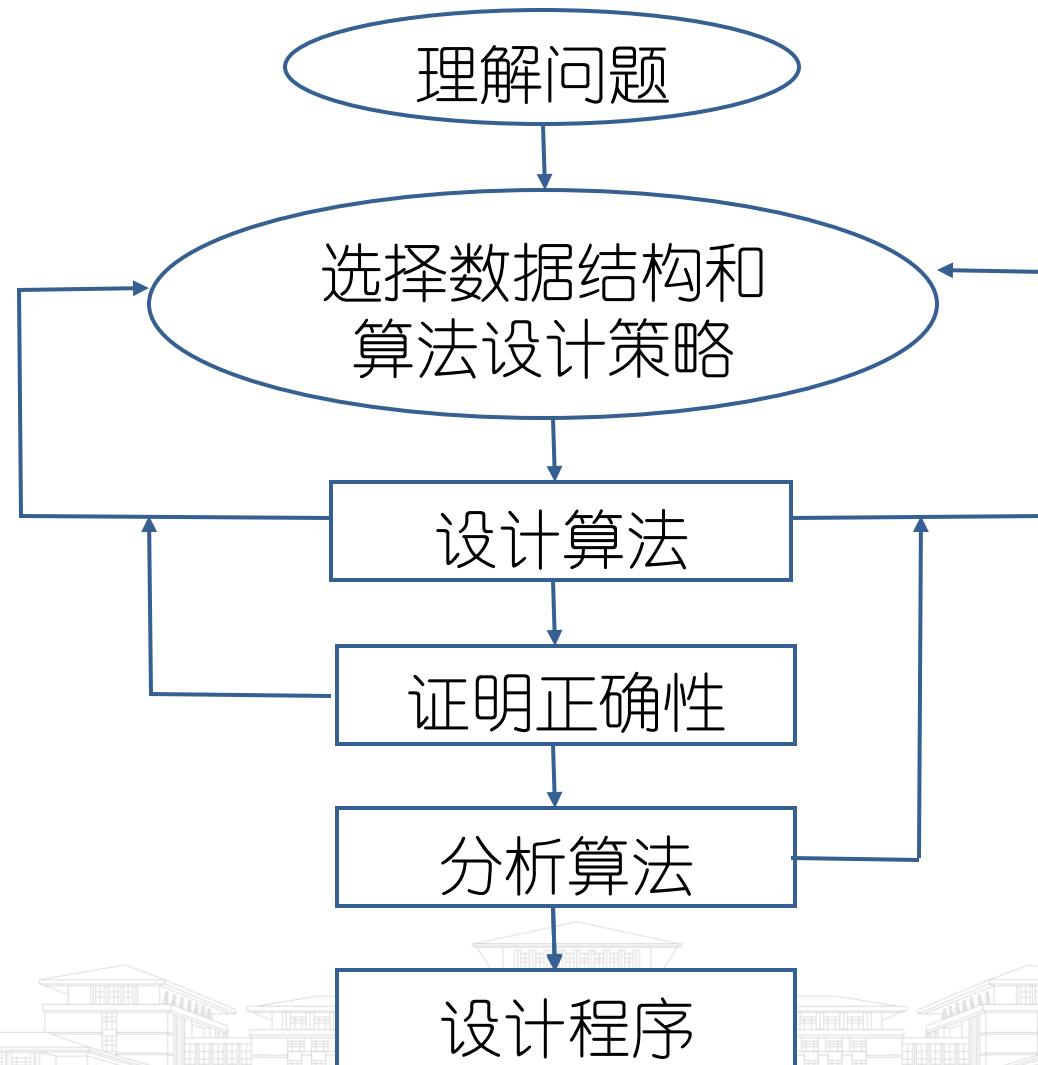


目录

- 1 什么是算法
- 2.1 算法学习的基本内容
- 2.2 算法的表示
- 2.3 算法的确认
- 2.4 算法的分析



计算问题的求解过程



2.1 算法学习的基本内容

- 如何设计算法
- 如何表示算法
- 如何确认算法
- 如何分析算法
- 如何测试程序



如何设计算法

- 面对具体问题，运用一些基本设计策略，规划算法。
 - 被实践证明是有用的
 - 计算机科学、运筹学、电器工程等领域
 - 设计出很多精致有效的好算法
- 掌握这些策略，设计出更多的好算法



如何表示算法

- 设计的算法要用恰当的方式地表示出来
- Code is Cheap, Describe It to Me!
- 程序伪代码——简单明了



如何确认算法

- 算法确认(**algorithm validation**)——证明该算法对所有可能的合法输入，都能给出正确答案
- 算法确认的目的——确保该算法能正确无误地工作
 - 穷举法
 - 推理——数学归纳法、反证法
 - 构造性证明
 - 测试



如何分析算法

- 分析执行一个算法时，
 - 占用多少CPU时间完成运算；
 - 占用多少存储器存储空间，存放程序和数据。
- 既对一个算法需要多少计算时间和存储空间，做定量分析。



测试程序

- 调试——调试只能指出有错误，而不能指出它们不存在错误。
 - 源于程序正确性的证明还没有取得突破性进展。
- 时空分布图
 - 用各种给定数据执行调试后的程序
 - 测定计算时间和空间
 - 印证之前的分析是否正确
 - 指出实现最优化的有效逻辑位置



目录

- 1 什么是算法
- 2.1 算法学习的基本内容
- 2.2 算法的表示
- 2.3 算法的确认
- 2.4 算法的分析



算法的表示

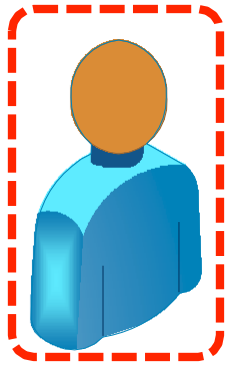
- 如何表示一个算法



算法的表示

- 如何表示一个算法
 - 自然语言

算法的设计者
依靠自然语言交流和表达



人类

机器



算法的表示

- 自然语言
 - 方法优势
 - 贴近人类思维，易于理解主旨
 - 不便之处
 - 语言描述繁琐，容易产生歧义
 - 使用了...等不严谨的描述



算法的表示

- 自然语言
 - 方法优势
 - 贴近人类思维，易于理解主旨
 - 不便之处
 - 语言描述繁琐，容易产生歧义
 - 使用了...等不严谨的描述

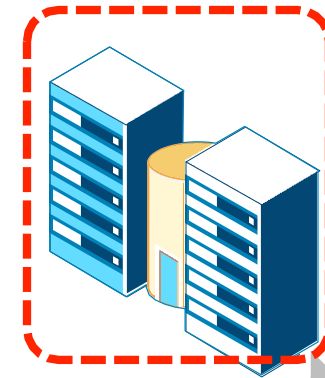
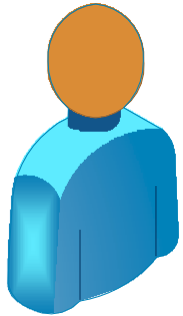
选择排序：

- 第一次遍历找到最小元素
- 第二次在剩余数组中遍历找到次小元素
- ...
- 第 n 次在剩余数组中遍历找到第 n 小元素

算法的表示

- 如何表示一个算法
 - 自然语言
 - 编程语言

算法的执行者
需要详细具体的执行代码



人类

机器



算法的表示

- 编程语言
 - 方法优势
 - 精准表达逻辑，规避表述歧义
 - 不便之处
 - 不同编程语言间语法存在差异
 - 过于关注算法实现的细枝末节



算法的表示

- 编程语言

- 方法优势

- 精准表达逻辑，规避表述歧义

- 不便之处

- 不同编程语言间语法存在差异
 - 过于关注算法实现的细枝末节

```
void select_sort(int*a,int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++){
            if(a[i] > a[j]){
                t=a[i];
                a[i]=a[j];
                a[j]=t;
            }
        }
    }
}
```

C语言

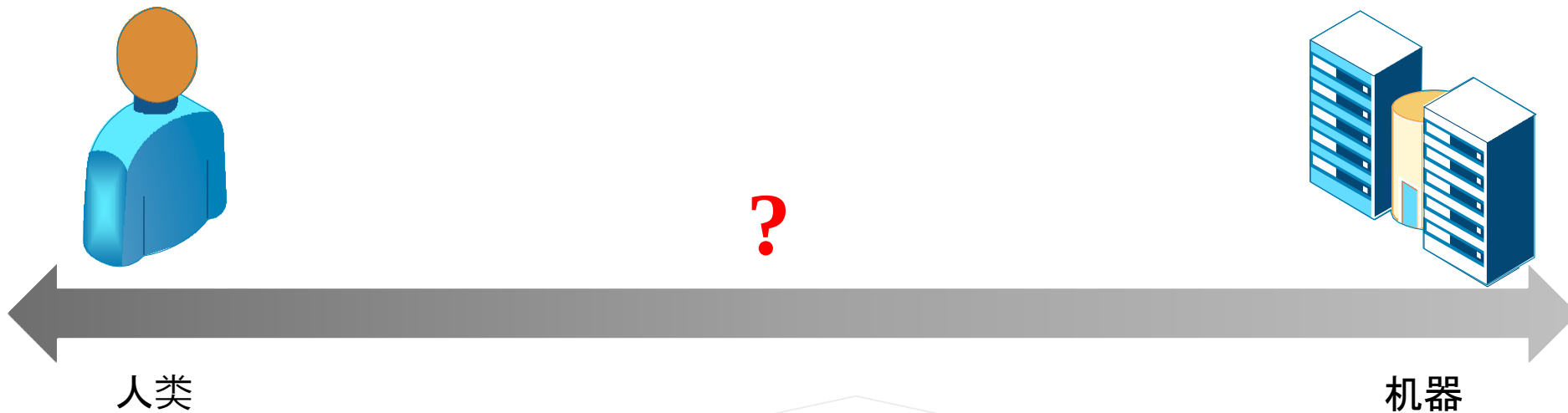
```
def select_sort(a, n):
    for i in range(0, n-1):
        for j in range(i+1,n):
            if a[i] > a[j]:
                tem = a[i]
                a[i] = a[j]
                a[j] = tem
```

Python语言



算法的表示

- 如何表示一个算法
 - 自然语言：贴近人类思维，易于理解主旨
 - 编程语言：精准表达逻辑，规避表述歧义



思考：可否同时兼顾两类表示方法的优势？

算法的表示

- 如何表示一个算法
 - 自然语言：贴近人类思维，易于理解主旨
 - 编程语言：精准表达逻辑，规避表述歧义



伪代码

- 非正式语言

- 移植**编程语言**书写形式作为基础和框架
- 按照接近**自然语言**的形式表达算法过程

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end  
end
```



伪代码

- 非正式语言

- 移植**编程语言**书写形式作为基础和框架
- 按照接近**自然语言**的形式表达算法过程

- 兼顾自然语言与编程语言优势

- **简洁**表达算法本质，不拘泥于实现细节
- **准确**反映算法过程，不产生矛盾和歧义

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```



选择排序伪代码

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

选择排序



选择排序伪代码

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

定义算法的输入和输出

```
for  $i \leftarrow 1$  to  $n - 1$  do
  for  $j \leftarrow i + 1$  to  $n$  do
    //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置
    if  $A[i] > A[j]$  then
      | 交换  $A[i]$  和  $A[j]$ 
    end
  end
end
end
```

选择排序



选择排序伪代码

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

 for $j \leftarrow i + 1$ to n do

 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

 if $A[i] > A[j]$ then

 | 交换 $A[i]$ 和 $A[j]$

 end

 end

end

循环
语句块缩进

选择排序



选择排序伪代码

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
  for  $j \leftarrow i + 1$  to  $n$  do  
    //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
    if  $A[i] > A[j]$  then  
      | 交换  $A[i]$  和  $A[j]$   
    end  
  end  
end
```

将 $i+1$ 赋值给 j

选择排序



选择排序伪代码

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

 for $j \leftarrow i + 1$ to n do

 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

 if $A[i] > A[j]$ then

 交换 $A[i]$ 和 $A[j]$

 end

 end

end

注释使用“//”符号

选择排序



选择排序伪代码

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

条件
语句块缩进

选择排序



选择排序伪代码

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

 for $j \leftarrow i + 1$ to n do

 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

 if $A[i] > A[j]$ then

 交换 $A[i]$ 和 $A[j]$

 end

 end

end

不关注交换过程的实现细节

选择排序

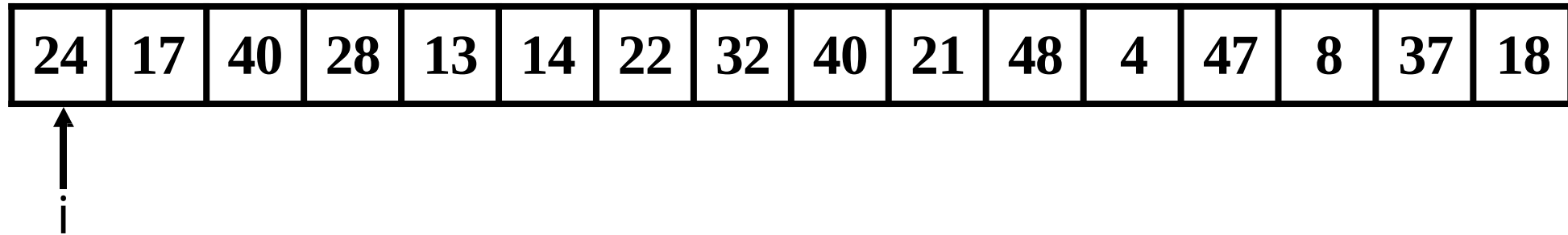


选择排序伪代码

24	17	40	28	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

从数组首部开始枚举 i

for $j \leftarrow i + 1$ to n do

//如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

if $A[i] > A[j]$ then

| 交换 $A[i]$ 和 $A[j]$

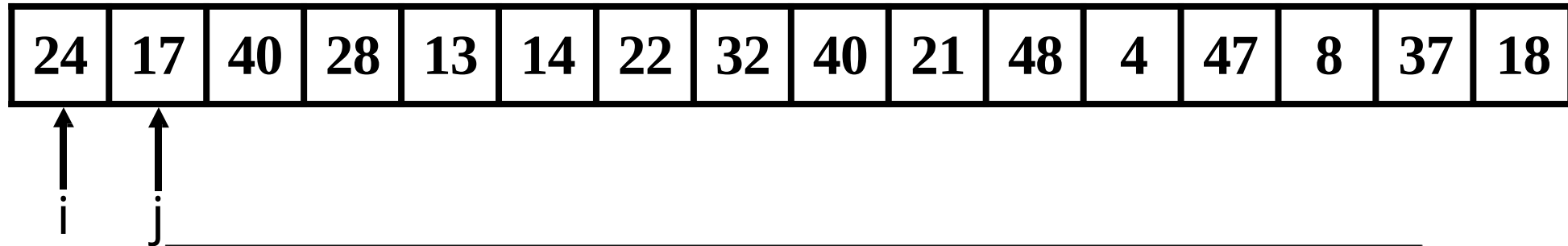
end

end

end

选择排序

选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

for $j \leftarrow i + 1$ to n do

//如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

if $A[i] > A[j]$ then

| 交换 $A[i]$ 和 $A[j]$

end

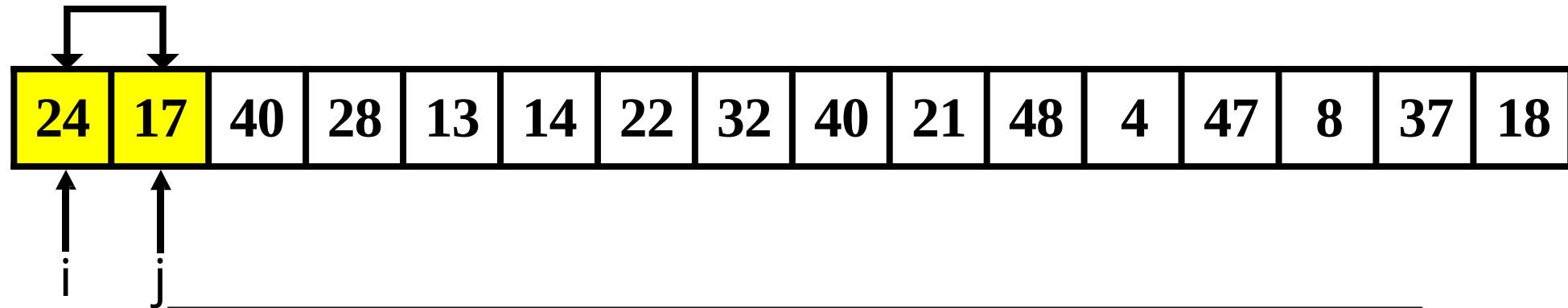
end

end

从 $i + 1$ 开始枚举 j

选择排序

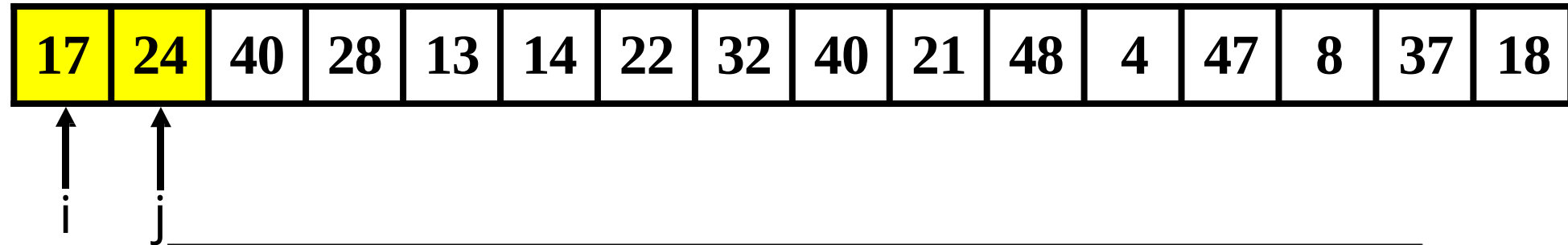
选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$
 输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
 for $i \leftarrow 1$ to $n - 1$ do
 for $j \leftarrow i + 1$ to n do
 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置
 if $A[i] > A[j]$ then
 交换 $A[i]$ 和 $A[j]$
 end
 end
 end

当第 i 个元素大于第 j 个元素时,
交换元素位置

选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

 for $j \leftarrow i + 1$ to n do

 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

 if $A[i] > A[j]$ then

 交换 $A[i]$ 和 $A[j]$

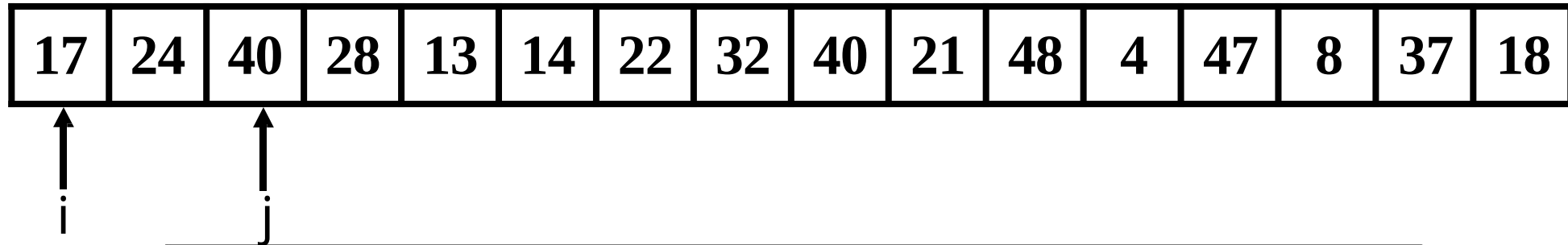
 end

 end

end

当第 i 个元素大于第 j 个元素时,
交换元素位置

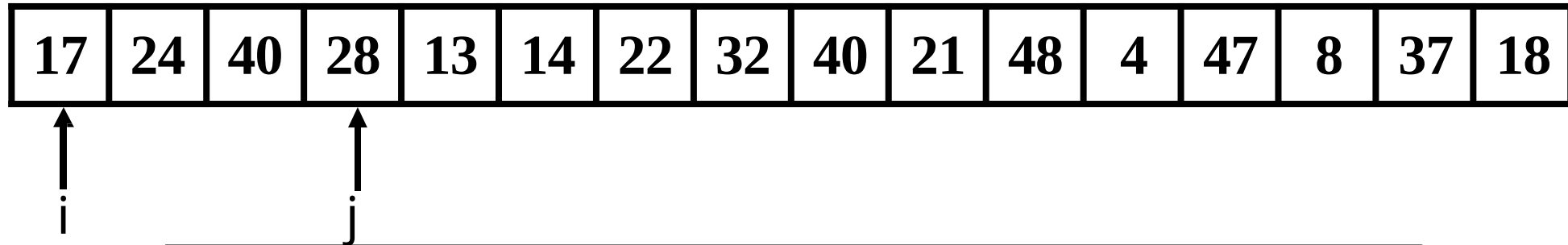
选择排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

继续枚举 j

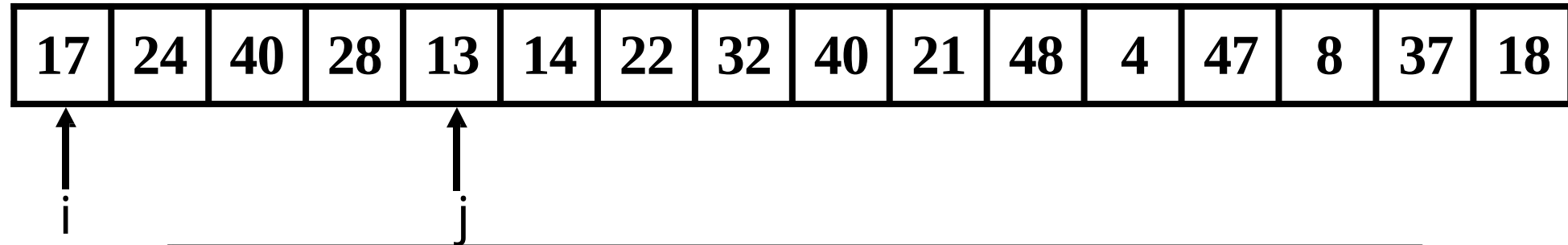
选择排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

继续枚举 j

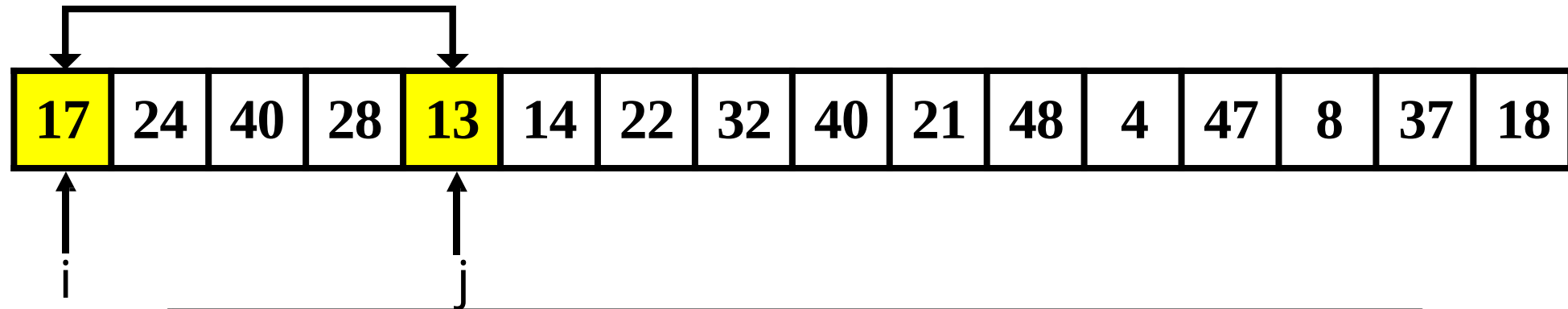
选择排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
  for  $j \leftarrow i + 1$  to  $n$  do  
    //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
    if  $A[i] > A[j]$  then  
      | 交换  $A[i]$  和  $A[j]$   
    end  
  end  
end  
end
```

继续枚举 j

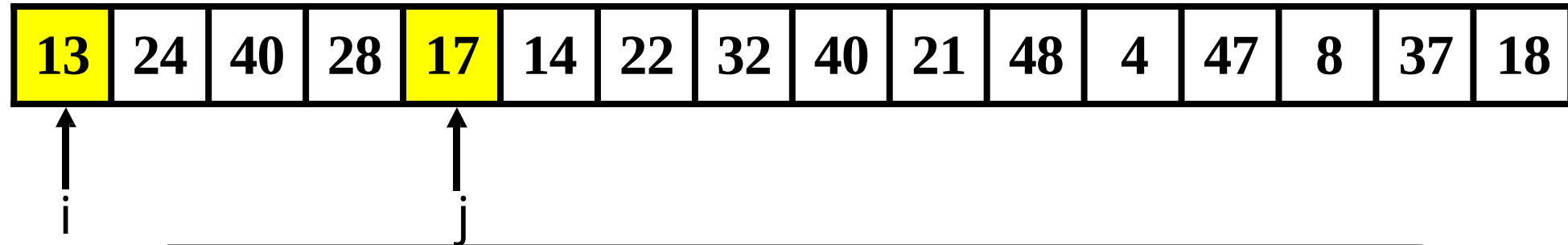
选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$
输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
for $i \leftarrow 1$ to $n - 1$ do
 for $j \leftarrow i + 1$ to n do
 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置
 if $A[i] > A[j]$ then
 交换 $A[i]$ 和 $A[j]$
 end
 end
end

当第 i 个元素大于第 j 个元素时,
交换元素位置

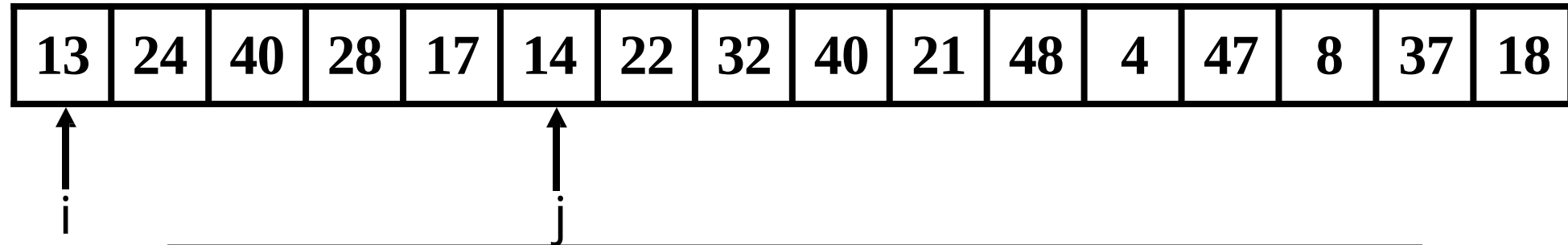
选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$
输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
for $i \leftarrow 1$ to $n - 1$ do
 for $j \leftarrow i + 1$ to n do
 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置
 if $A[i] > A[j]$ then
 交换 $A[i]$ 和 $A[j]$
 end
 end
end

当第 i 个元素大于第 j 个元素时,
交换元素位置

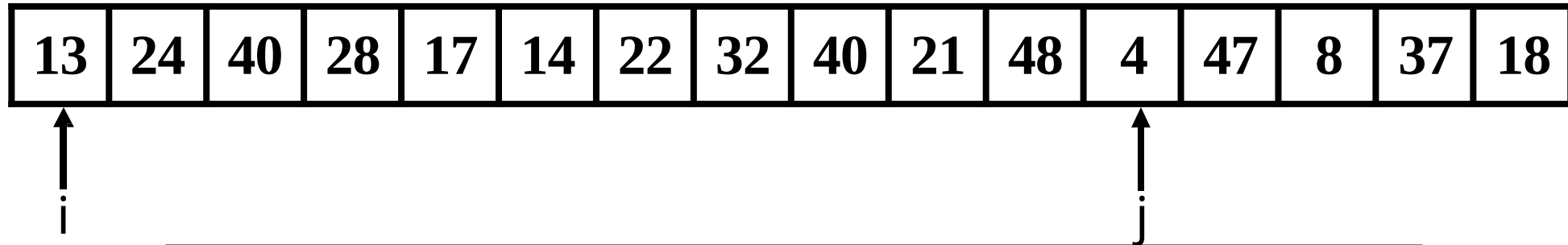
选择排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

继续枚举 j

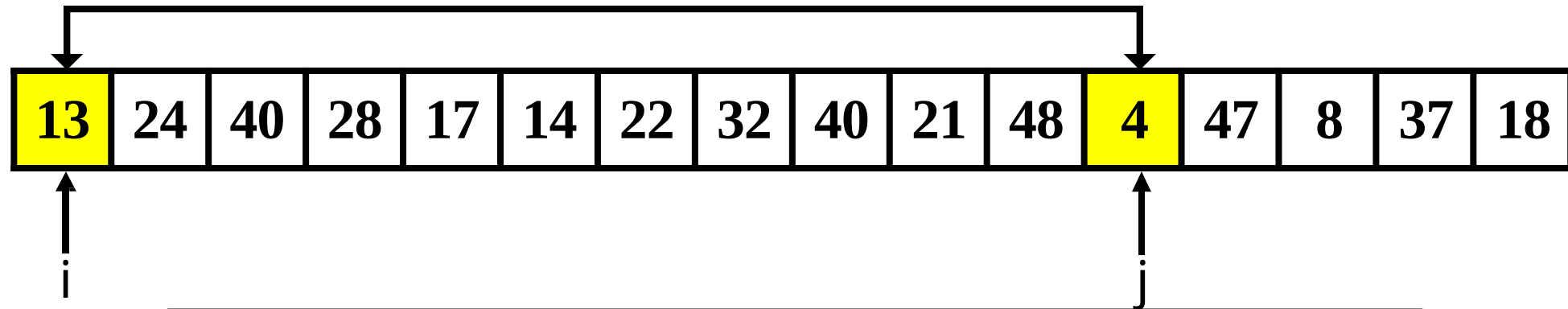
选择排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

继续枚举 j

选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

 for $j \leftarrow i + 1$ to n do

 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

 if $A[i] > A[j]$ then

 交换 $A[i]$ 和 $A[j]$

 end

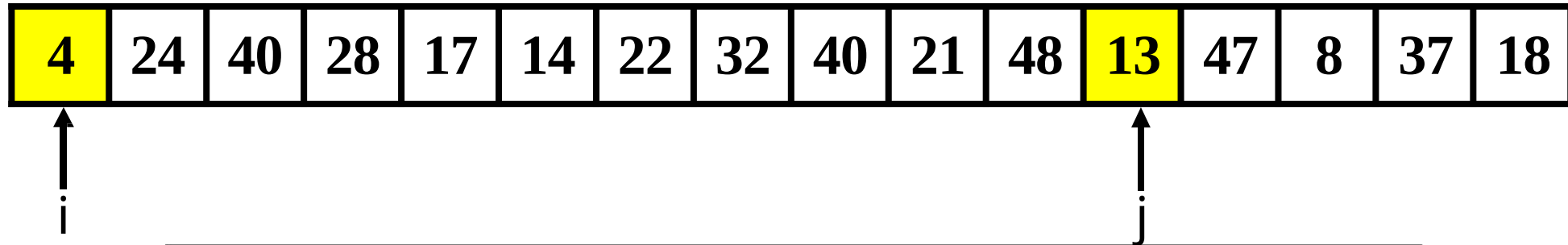
 end

end

当第 i 个元素大于第 j 个元素时,
交换元素位置

选择排序

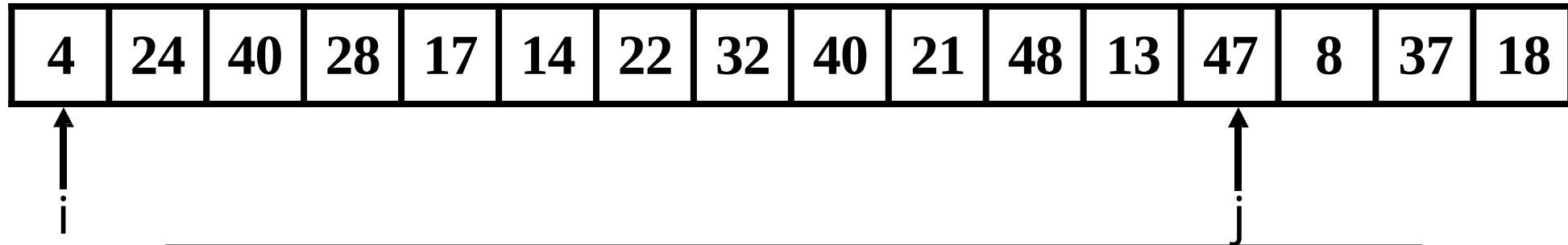
选择排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$
输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
for $i \leftarrow 1$ to $n - 1$ do
 for $j \leftarrow i + 1$ to n do
 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置
 if $A[i] > A[j]$ then
 交换 $A[i]$ 和 $A[j]$
 end
 end
end

当第 i 个元素大于第 j 个元素时,
交换元素位置

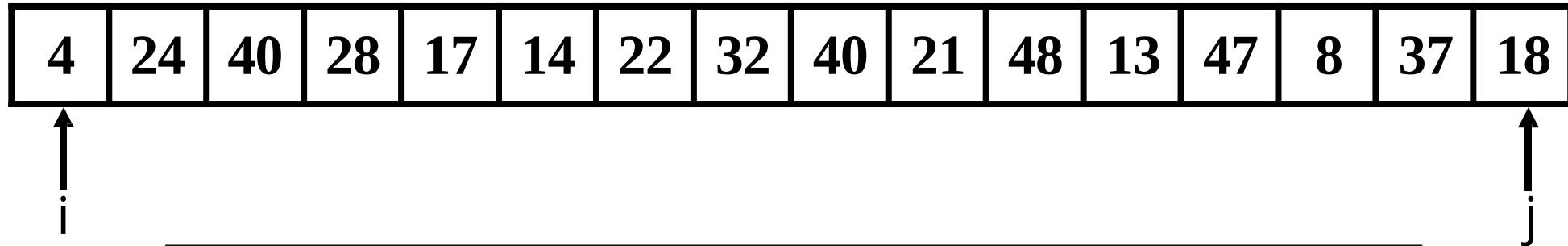
选择排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

继续枚举 j

选择排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```

选择排序伪代码

4	24	40	28	17	14	22	32	40	21	48	13	47	8	37	18
---	----	----	----	----	----	----	----	----	----	----	----	----	---	----	----

↑
 i

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

 for $j \leftarrow i + 1$ to n do

 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

 if $A[i] > A[j]$ then

 | 交换 $A[i]$ 和 $A[j]$

 end

 end

end

继续枚举 i

选择排序

选择排序伪代码

4	8	40	28	24	17	22	32	40	21	48	14	47	13	37	18
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

↑
 i

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

 for $j \leftarrow i + 1$ to n do

 //如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

 if $A[i] > A[j]$ then

 | 交换 $A[i]$ 和 $A[j]$

 end

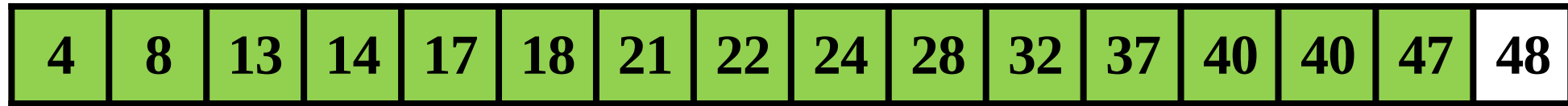
 end

end

继续枚举 i

选择排序

选择排序伪代码



.....

↑
 i

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $i \leftarrow 1$ to $n - 1$ do

枚举 i 至 $n - 1$, 结束循环

for $j \leftarrow i + 1$ to n do

//如果 $A[i]$ 大于 $A[j]$, 则交换二者位置

if $A[i] > A[j]$ then

| 交换 $A[i]$ 和 $A[j]$

end

end

end

.....

选择排序

选择排序伪代码

4	8	13	14	17	18	21	22	24	28	32	37	40	40	47	48
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $i \leftarrow 1$  to  $n - 1$  do  
    for  $j \leftarrow i + 1$  to  $n$  do  
        //如果  $A[i]$  大于  $A[j]$ , 则交换二者位置  
        if  $A[i] > A[j]$  then  
            | 交换  $A[i]$  和  $A[j]$   
        end  
    end  
end
```


插入排序伪代码

17	24	28	40	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

插入排序伪代码

key=13

17	24	28	40	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

j

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $j \leftarrow 2$ to n do

$key \leftarrow A[j]$

 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中

$i \leftarrow j - 1$

 while $i > 0$ and $A[i] > key$ do

$A[i+1] \leftarrow A[i]$

$i \leftarrow i - 1$

 end

$A[i+1] \leftarrow key$

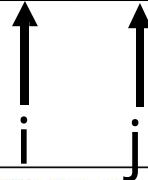
end

将 $A[j]$ 赋值给 key

插入排序伪代码

key=13

17	24	28	40	13	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----



输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $j \leftarrow 2$ to n do

$key \leftarrow A[j]$

 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中

$i \leftarrow j - 1$

 while $i > 0$ and $A[i] > key$ do

$A[i+1] \leftarrow A[i]$

$i \leftarrow i - 1$

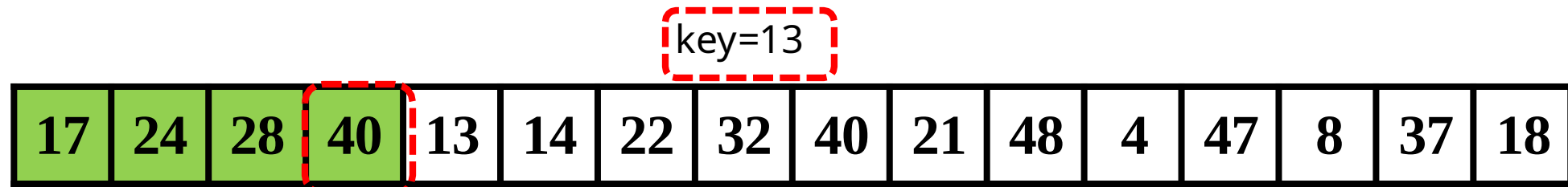
 end

$A[i+1] \leftarrow key$

end

从j-1开始枚举i

插入排序伪代码

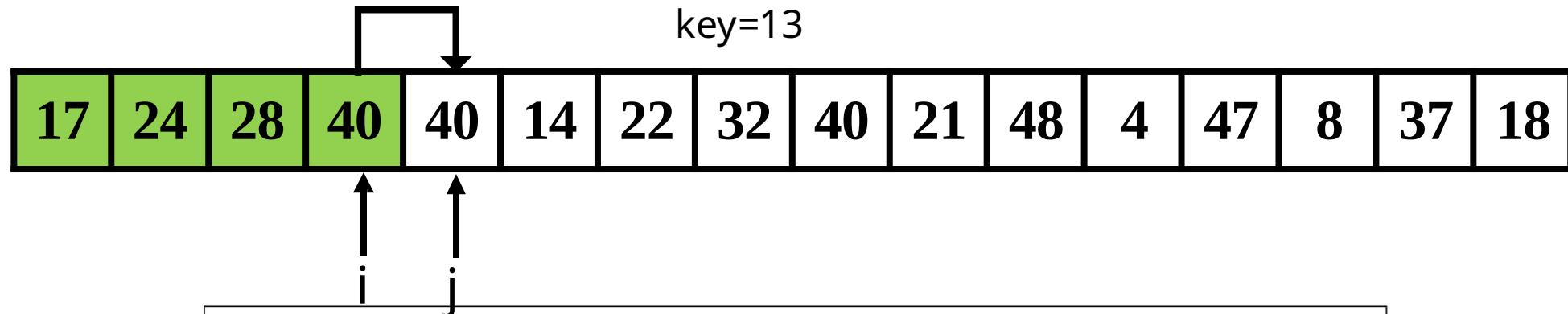


$i > 0$
 $A[i] > \text{key}$

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $\text{key} \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > \text{key}$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow \text{key}$   
end
```

判断循环条件

插入排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$
输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
for $j \leftarrow 2$ to n do
 $key \leftarrow A[j]$
 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > key$ do
 $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 end
 $A[i+1] \leftarrow key$
end

将 $A[i]$ 赋值给 $A[i+1]$

插入排序伪代码

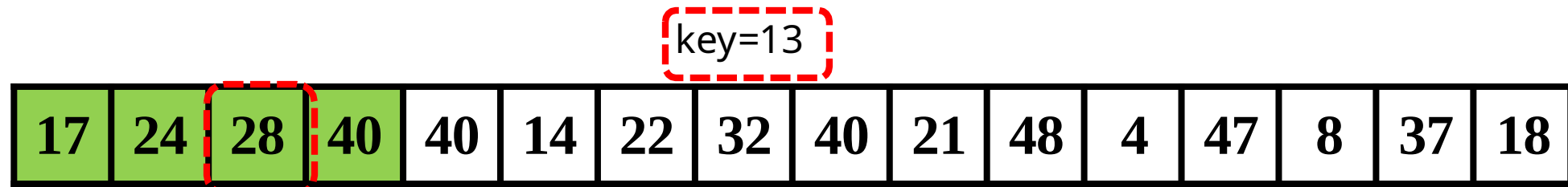
key=13

17	24	28	40	40	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

输入: 数组 $A[a_1, a_2, \dots, a_n]$
输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
for $j \leftarrow 2$ to n do
 $key \leftarrow A[j]$
 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中
 $i \leftarrow j-1$
 while $i > 0$ and $A[i] > key$ do
 $A[i+1] \leftarrow A[i]$
 $i \leftarrow i-1$
 end
 $A[i+1] \leftarrow key$
end

将i左移一步

插入排序伪代码

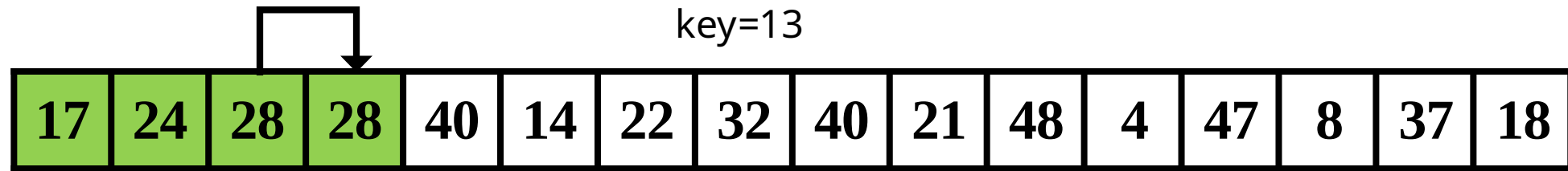


$i > 0$
 $A[i] > \text{key}$

输入: 数组 $A[a_1, a_2, \dots, a_n]$
输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
for $j \leftarrow 2$ to n do
 $\text{key} \leftarrow A[j]$
 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > \text{key}$ do
 $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 end
 $A[i+1] \leftarrow \text{key}$
end

判断循环条件

插入排序伪代码



输入: 数组 $A[a_1, a_2, \dots, a_n]$
 输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
 for $j \leftarrow 2$ to n do
 $key \leftarrow A[j]$
 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > key$ do
 $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 end
 $A[i+1] \leftarrow key$
 end

将 $A[i]$ 赋值给 $A[i+1]$

插入排序伪代码

key=13

17	24	28	28	40	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

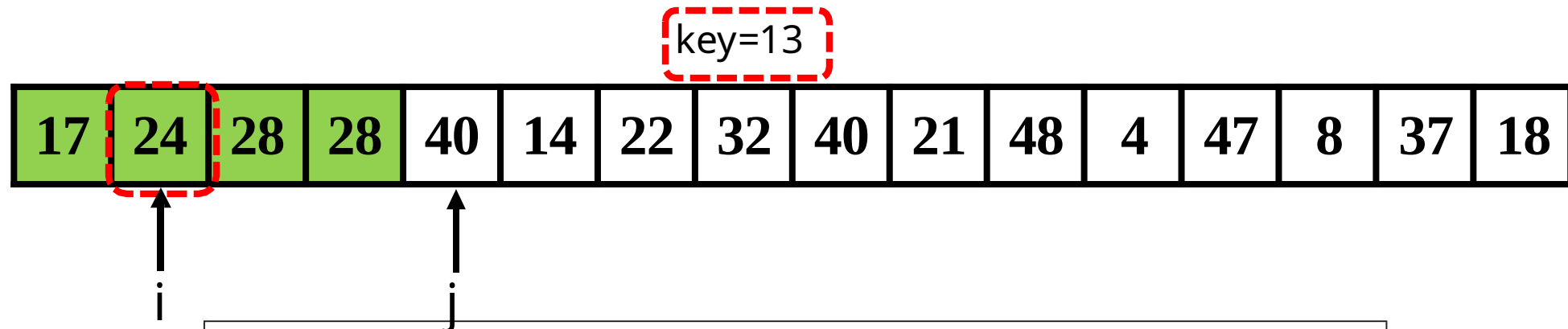
↑
 i

↑
 j

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

将 i 左移一步

插入排序伪代码

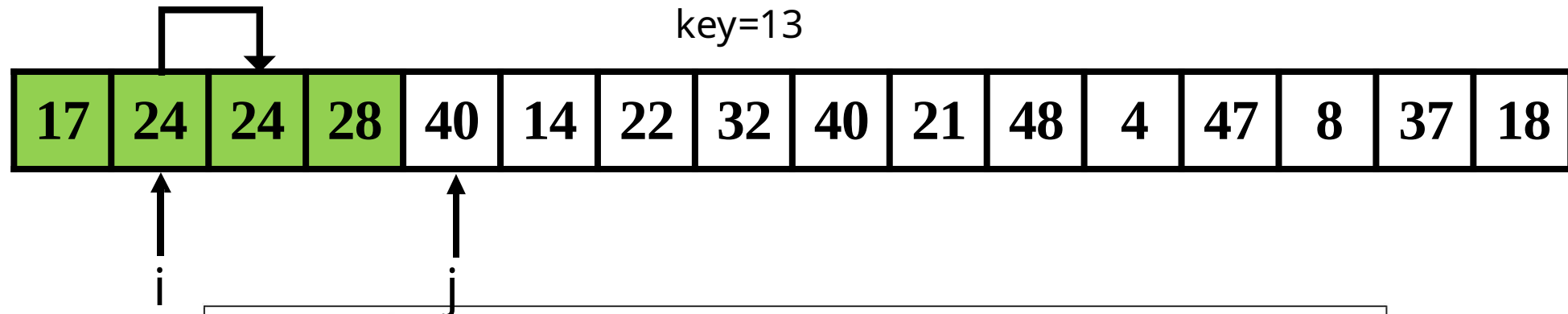


$i > 0$
 $A[i] > \text{key}$

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $\text{key} \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > \text{key}$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow \text{key}$   
end
```

判断循环条件

插入排序伪代码

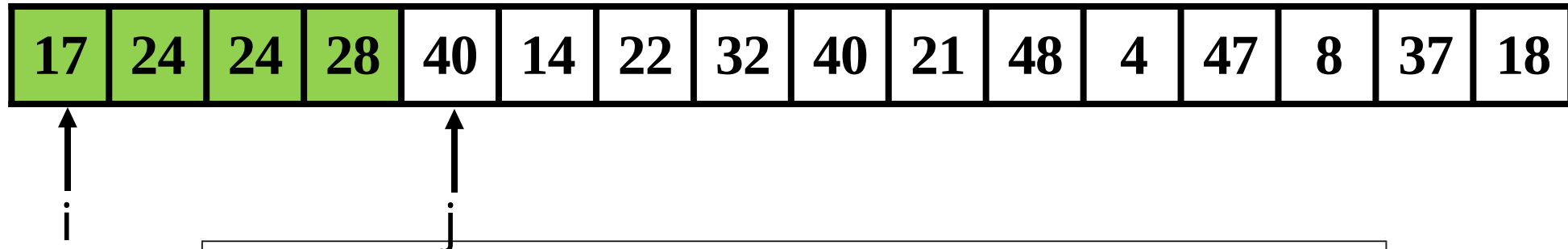


输入: 数组 $A[a_1, a_2, \dots, a_n]$
输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
for $j \leftarrow 2$ to n do
 $key \leftarrow A[j]$
 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > key$ do
 $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 end
 $A[i+1] \leftarrow key$
end

将 $A[i]$ 赋值给 $A[i+1]$

插入排序伪代码

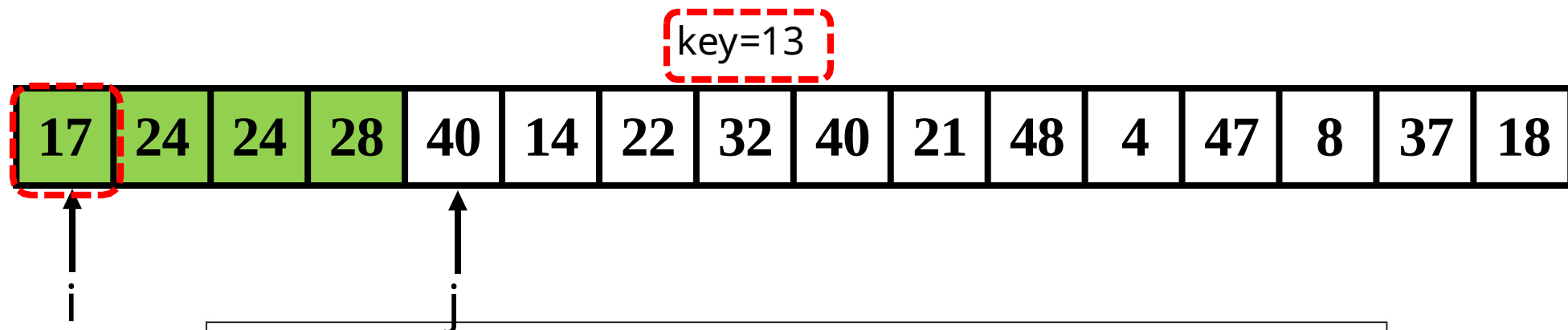
key=13



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

将 i 左移一步

插入排序伪代码

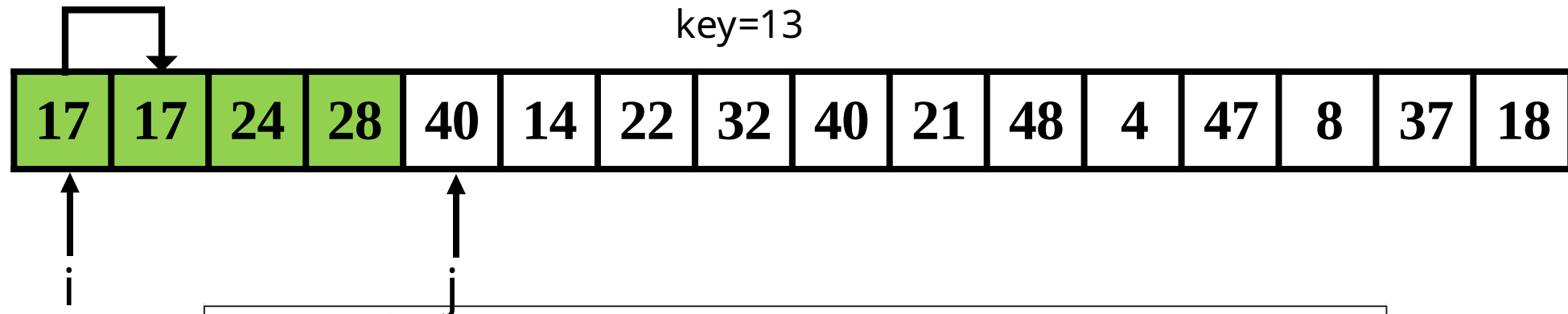


$i > 0$
 $A[i] > \text{key}$

输入: 数组 $A[a_1, a_2, \dots, a_n]$
 输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$
 for $j \leftarrow 2$ to n do
 $\text{key} \leftarrow A[j]$
 //将 $A[j]$ 插入到已排序的数组 $A[1..j-1]$ 中
 $i \leftarrow j - 1$
 while $i > 0$ and $A[i] > \text{key}$ do
 $A[i+1] \leftarrow A[i]$
 $i \leftarrow i - 1$
 end
 $A[i+1] \leftarrow \text{key}$
 end

判断循环条件

插入排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

将 $A[i]$ 赋值给 $A[i+1]$

插入排序伪代码

key=13

17	17	24	28	40	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

↑
i

↑
j

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

将i左移一步

插入排序伪代码

key=13

17	17	24	28	40	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

↑
i

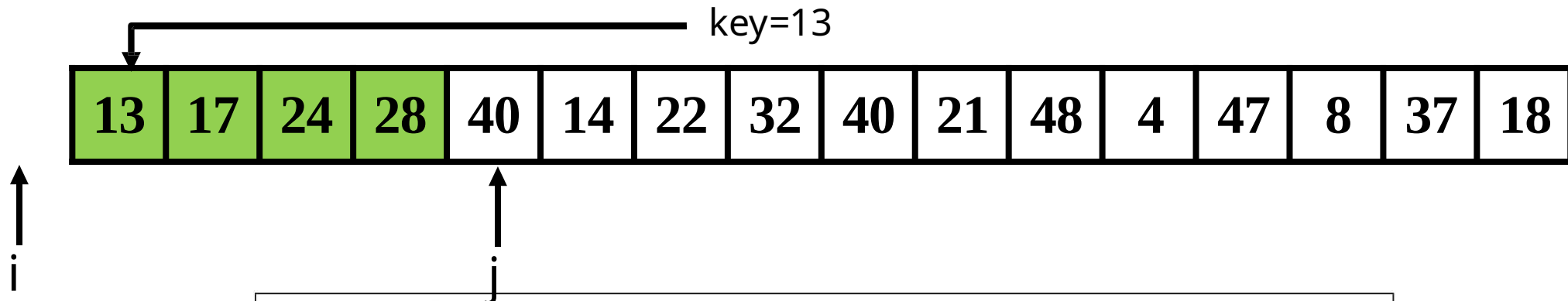
↑
j

i = 0
不满足循环条件

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

判断循环条件

插入排序伪代码



```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

将key赋值给A[i+1]

插入排序伪代码

13	17	24	28	40	14	22	32	40	21	48	4	47	8	37	18
----	----	----	----	----	----	----	----	----	----	----	---	----	---	----	----

```
输入: 数组  $A[a_1, a_2, \dots, a_n]$   
输出: 升序数组  $A'[a'_1, a'_2, \dots, a'_n]$ , 满足  $a'_1 \leq a'_2 \leq \dots \leq a'_n$   
for  $j \leftarrow 2$  to  $n$  do  
     $key \leftarrow A[j]$   
    //将  $A[j]$  插入到已排序的数组  $A[1..j-1]$  中  
     $i \leftarrow j - 1$   
    while  $i > 0$  and  $A[i] > key$  do  
         $A[i+1] \leftarrow A[i]$   
         $i \leftarrow i - 1$   
    end  
     $A[i+1] \leftarrow key$   
end
```

目录

- 1 什么是算法
- 2.1 算法学习的基本内容
- 2.2 算法的表示
- 2.3 算法的确认
- 2.4 算法的分析



算法的确认

- 如何证明算法的正确性？
- 使用“**循环不变式**”（Loop Invariant）来证明**增量法**的正确性
 - **初始化**：循环的第一次迭代之前，它为真；
 - **保持**：如果循环的某次迭代之前它为真，那么下一次迭代之前它仍为真；
 - **终止**：在循环终止时，不变式为我们提供一个有用的性质，该性质有助于证明算法是正确的
- 本质：**数学归纳法**（Mathematics Induction）



插入排序算法的确认

- 循环不变式：
 - 在 **for** 循环每次迭代开始之前，子数组 $A[1 \dots j - 1]$ 由原来在 $A[1 \dots j - 1]$ 中的元素组成，且已经排好序
- 证明：
 - **初始化**：第一次迭代开始前 $j = 2$ ， $A[1]$ 天然有序且是原来元素
 - **保持**：若迭代开始前， $A[1 \dots j - 1]$ 有序且由原本元素组成；迭代过程中将 $A[j - 1]$, $A[j - 2]$, $A[j - 3]$ 等右移，直到空出合适的位置放 $A[j]$ ；所以下次迭代开始前， $A[1 \dots j]$ 有序且由原本元素组成
 - **终止**：循环终止的时候 $j = n + 1$ ，循环不变式告诉我们 $A[1 \dots n]$ 有序且由原本元素组成。



目录

- 1 什么是算法
- 2.1 算法学习的基本内容
- 2.2 算法的表示
- 2.3 算法的确认
- 2.4 算法的分析



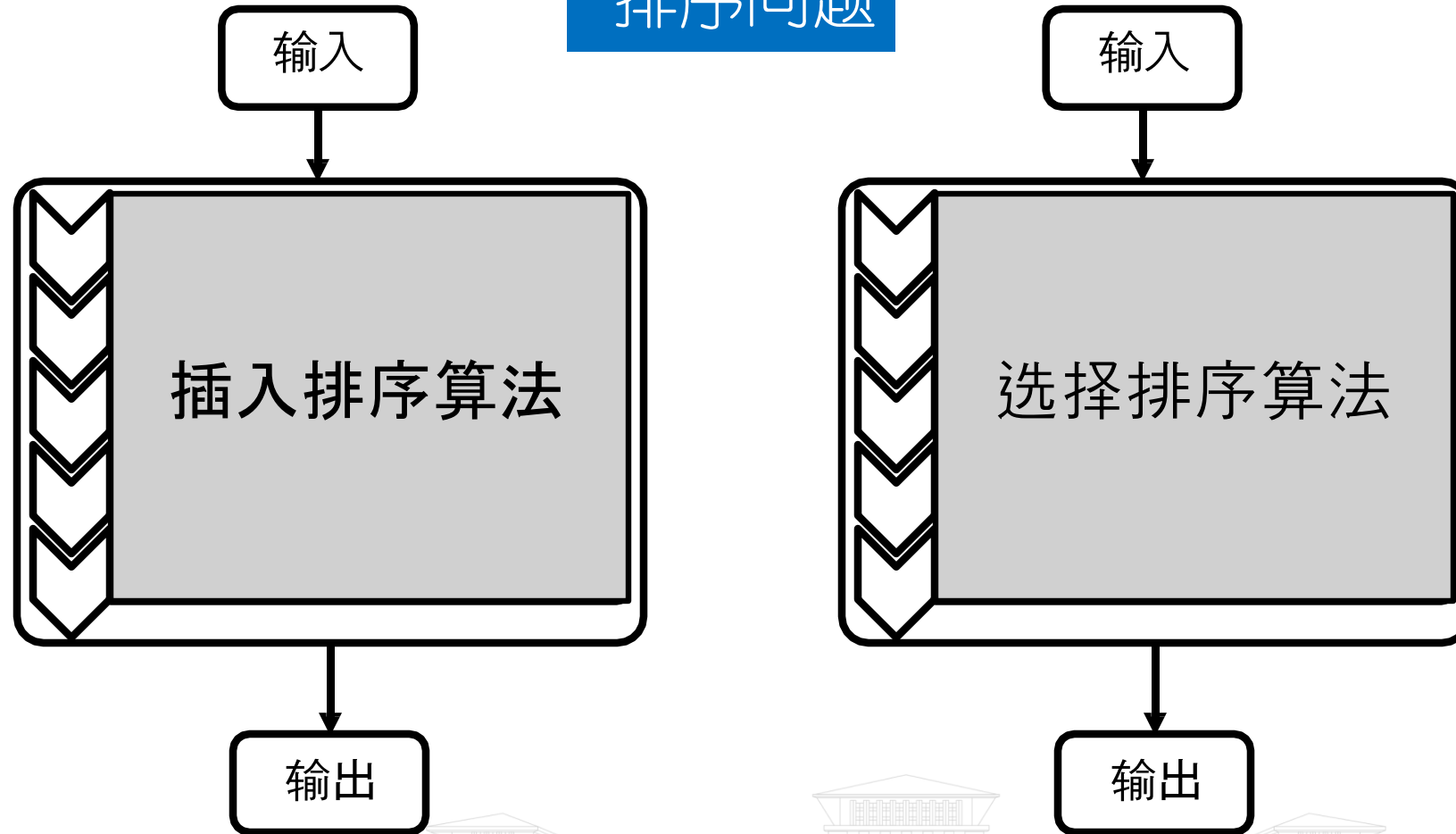
算法的分析

- 算法分析目的
- 算法分析的准备工作
- 计算时间的渐进表示
- 一些证明方法
- 作时空性能分布图



算法分析的目的

排序问题



思考：如何比较不同算法性能？

算法分析的目的

- 算法分析(analysis of algorithms)是对一个算法需要多少**计算时间**和**存储空间**作定量的分析。
——确定算法在什么样的环境下能够有效地运行。



算法运行假定的计算机类型

- 要求独立于具体的软硬件环境单纯分析算法。
- 假定使用一台通用计算机
 - 顺序处理每条指令；
 - 存储容量足够大；
 - 存取时间恒定。



算法分析的目的

- 算法分析(analysis of algorithms)是对一个算法需要多少**计算时间**和**存储空间**作定量的分析。
 - 确定算法在什么样的环境下能够有效地运行。
- 分析在**最好**、**最坏**和**平均**情况下的执行情况。
 - 对同一问题不同算法的有效性作出比较。



实例影响算法运行时间

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

for $j \leftarrow 2$ to n do

$key \leftarrow A[j]$

$i \leftarrow j - 1$

 while $i > 0$ and $A[i] > key$ do

$A[i + 1] \leftarrow A[i]$

$i \leftarrow i - 1$

 end

$A[i + 1] \leftarrow key$

end

循环次数未知

插入排序算法伪代码



实例影响算法运行时间

- 插入排序最好情况：数组升序

- 比较次数： $1+1+1+\dots+1 = n-1$

4	8	13	14	17	18	21	22	24	28	32	37	40	40	47	48
---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

- 插入排序最坏情况：数组降序

- 比较次数： $1+2+3+\dots+n-1 = n(n-1)/2$

48	47	40	40	37	32	28	24	22	21	18	17	14	13	8	4
----	----	----	----	----	----	----	----	----	----	----	----	----	----	---	---



时间复杂性的形式化定义

- 算法的时间复杂性 $T(n)$;
- 算法的空间复杂性 $S(n)$;
- 其中 n 是问题的规模。

最坏情况下: $T_{\max}(n) = \max\{ T(I) \mid \text{size}(I)=n \}$

最好情况下: $T_{\min}(n) = \min\{ T(I) \mid \text{size}(I)=n \}$

平均情况下: $T_{\text{avg}}(n) = \sum_{\text{size}(I)=n} p(I)T(I)$

其中 I 是问题的规模为 n 的实例,

$p(I)$ 是实例 I 出现的概率。



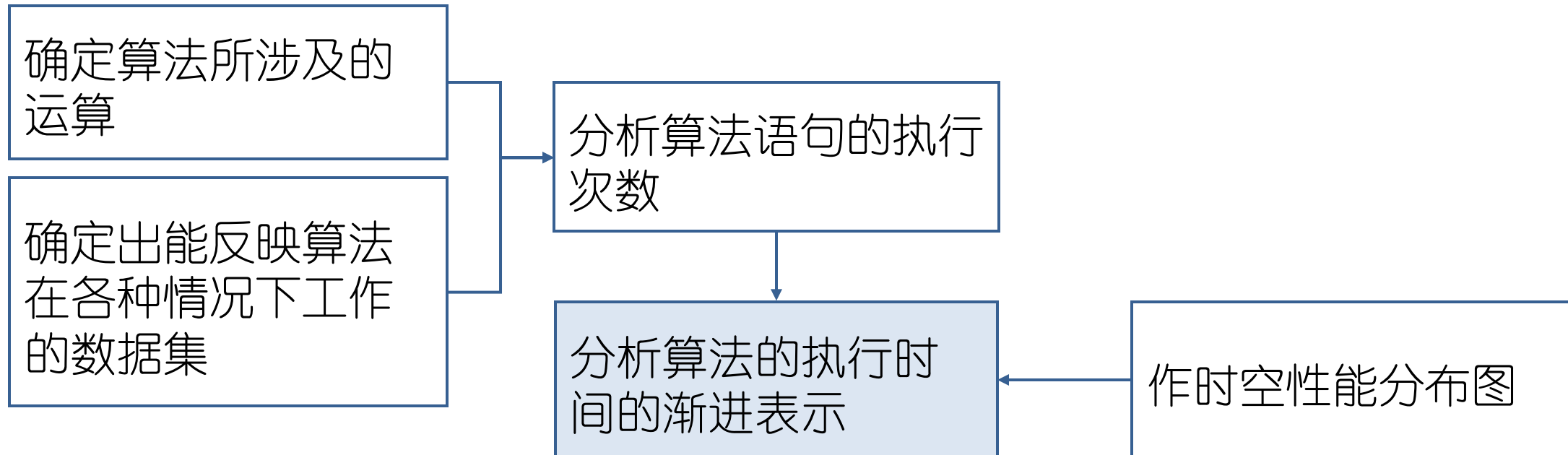
算法分析过程

全面分析的两个阶段

准备工作

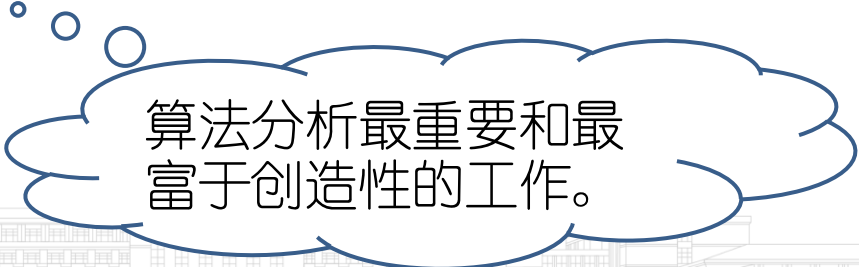
事前分析

事后测试

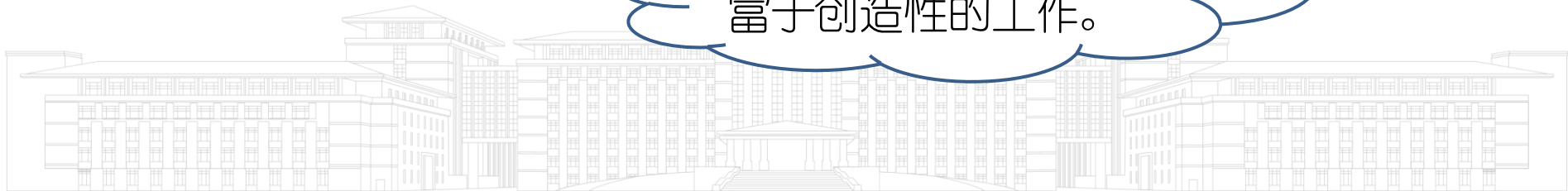


准备工作

- 首先确定使用哪些**运算**以及执行这些运算所用的时间。
 - 基本运算
 - 由一些更基本的任意长序列的运算所组成的复杂运算
- 其次是要确定出能反映算法在各种情况下工作的数据集。
 - 编造出能产生**最好**、**最坏**和**有代表性**情况的数据配置
 - 通过使用这些数据来运行算法，以更好地了解算法的性能



算法分析最重要和最富于创造性的工作。



基本运算

- 时间囿界于常数的运算
 - 加、减、乘、除整数算术运算
 - 浮点算术、字符比较、对变量赋值、过程调用等
- 每种运算所用时间虽然不同，但都只花费一个固定量的时间



复杂运算

- 由一些更基本的任意长序列的运算组成，如：两个字符串的比较运算
 - 一系列字符比较指令
 - 一个字符的比较时间——囿界于常数
 - 字符串比较的时间总量则取决于字符串的长度



全面分析算法的两个阶段

- 事前分析(a priori analysis)
 - 确定每条语句的执行次数
 - 求出该算法的一个时间限界函数(关于问题规模 n 的函数)
- 事后测试(a posteriori testing)
 - 作时空性能分布图



算法的执行时间

- 同一条语句在一个算法中的执行次数 (frequency count)称为频率计数
 - 由算法直接确定，与所用的机器无关，且独立于程序设计语言。
- 语句的时间总量=频率计数 × 执行一次该语句所需要的时间
 - 语句本质上是由运算组成的
 - 语句的执行时间依赖于机器、程序设计语言、编译程序
- 算法的执行时间就是构成算法的所有语句的执行时间总量之和



例：计算语句 $x \leftarrow z + y$ 在下面三个程序段中的频率计数

程序段1

begin

$x \leftarrow z + y$

end

1次

程序段2

begin

for $i \leftarrow 1$ to n do

$x \leftarrow z + y$

end

n 次

程序段3

begin

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

$x \leftarrow z + y$

end

n^2 次

- 语句的数量级是指执行它的频率
- 算法的数量级是指算法的所有语句的执行频率之和

确定一个算法的数量级十分重要，因为它在本质上反映了算法所需的计算时间。



计算时间的基本特性

● 插入排序最坏情况

```

for j ← 2 to n do ..... n次
    key ← A[j] ..... n-1次
    i ← j - 1 ..... n-1次
    while i > 0 and A[i] > key do...  $\sum_{k=2}^n k$ 次
        | A[i+1] ← A[i] .....  $\sum_{k=2}^n k-1$ 次
        | i ← i - 1 .....  $\sum_{k=2}^n k-1$ 次
    end
    A[i+1] ← key ..... n-1次
end
    
```

求和

$$T(n) = \frac{3}{2}n^2 + \frac{7}{2}n - 4$$

● 选择排序最坏情况

```

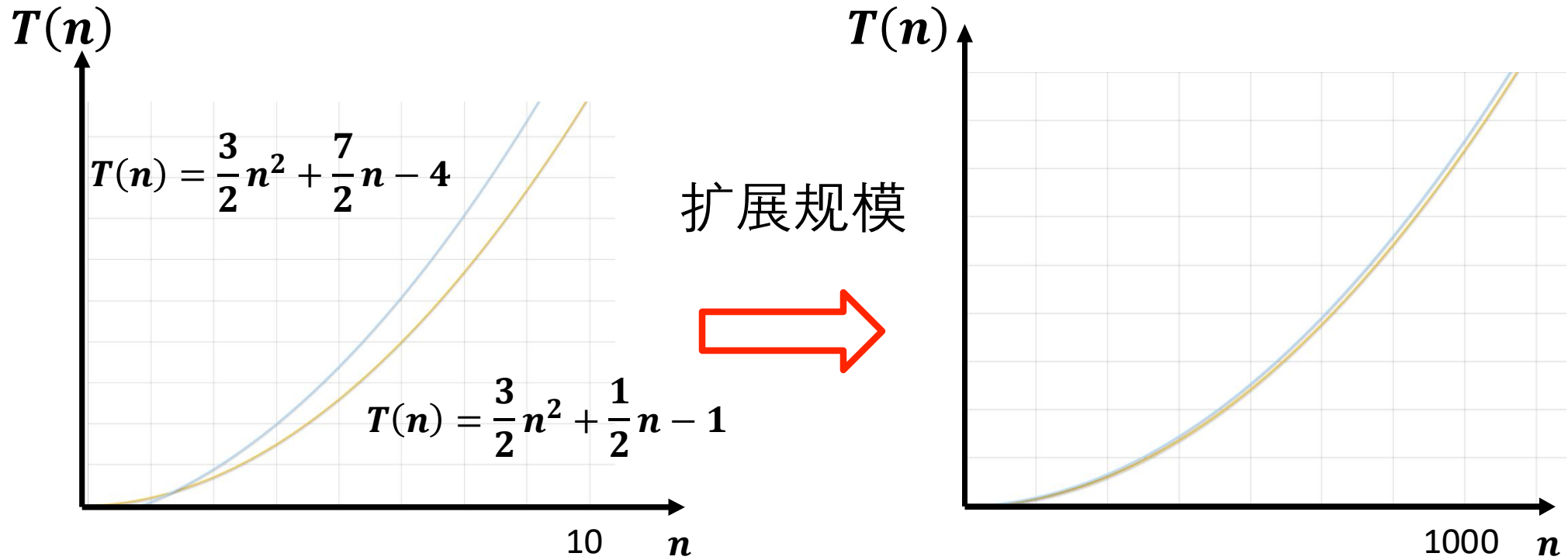
for i ← 1 to n-1 do ..... n次
    for j ← i+1 to n do...  $\sum_{k=0}^{n-2} n-k$ 次
        | if A[i] > A[j] then..  $\sum_{k=1}^{n-1} n-k$ 次
        | | 交换 A[i] 和 A[j] .....  $\sum_{k=1}^{n-1} k$ 次
        | end
    end
end
    
```

求和

$$T(n) = \frac{3}{2}n^2 + \frac{1}{2}n - 1$$

问题：能否简洁地衡量算法运行时间？

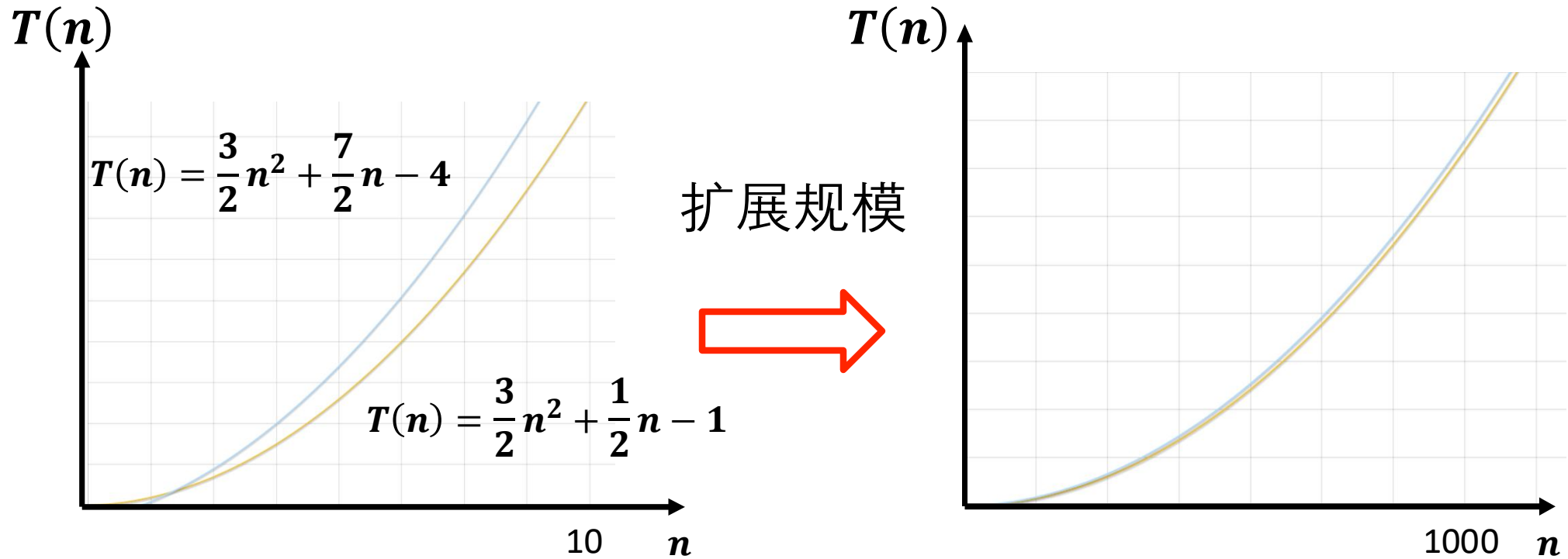
计算时间的基本特性



- 在 n 充分大时，两者相差不大
- 原因？



计算时间的基本特性



- 在 n 充分大时，两者相差不大
- 原因：两函数的最高阶项相同



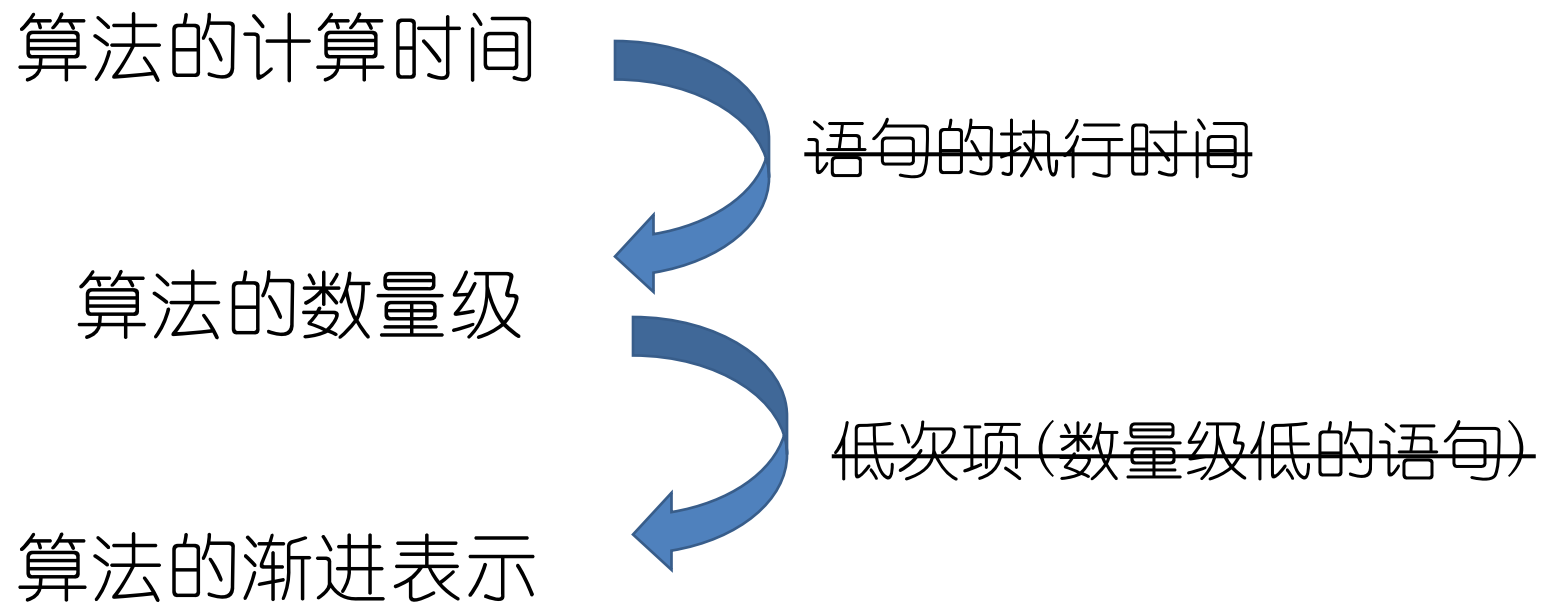
计算时间的基本特性

- 描述算法数量级的多项表达式 ×
- 最高次项 ×
- 最高次项的系数 ×
- 最高次项的次数 $\checkmark$

准确分析出算法数量级的多项式表达式是很困难的，因此我们对事前分析的计算时间进行渐进表示。



总结



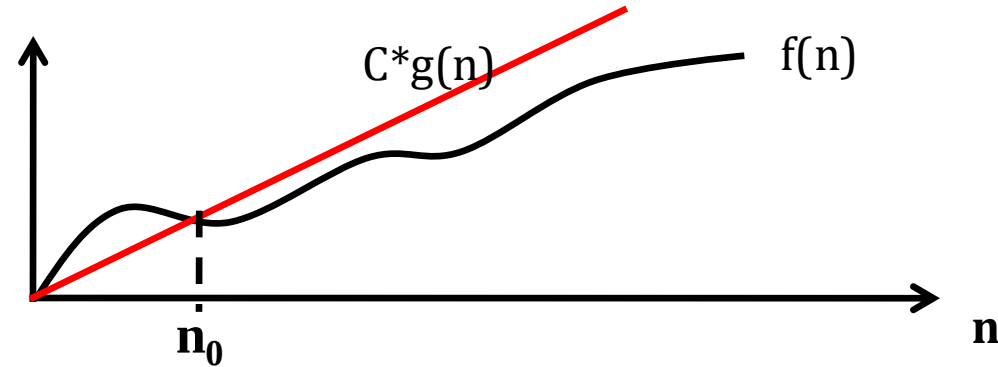
计算时间的渐进表示

- 定义2.1: $f(n) = O(g(n))$
- 定义2.2: $f(n) = \Omega(g(n))$
- 定义2.3: $f(n) = \Theta(g(n))$
- 定理2.1
- n 表示问题规模,
 - 输入或输出量;
 - 两者之和;
 - 其中之一的某种测度。
- $f(n)$ 表示算法的计算时间。
- $g(n)$ 是在事前分析中确定的某个形式简单的函数。

$f(n)$ 与机器和语言有关, 而 $g(n)$ 是独立于机器和语言的。



定义2.1



如果存在两个正常数 c 和 n_0 ，对于所有的 $n \geq n_0$ ，有 $|f(n)| \leq c|g(n)|$ ，则记作： $f(n) = O(g(n))$ 。

- 当 n 充分大时， $f(n)$ 有上界，一个常数倍的 $g(n)$ 是 $f(n)$ 的一个上界， $f(n)$ 的数量级就是 $g(n)$ 。
- $f(n)$ 的阶不高于 $g(n)$ 的阶。
- 在确定 $f(n)$ 的数量级时，总是试图求出最小的 $g(n)$ 。
- 有关证明中，找出 c 和 n_0 是关键。

判断 $f(n) = O(g(n))$?

基于定义证明你的判断

• $f(n) = 3n$, $g(n) = n$ 证明: $\exists c=3, n_0=1, \forall n \geq n_0$, 均有 $3n \leq cn$ 。证毕。

• $f(n) = n+1024$, $g(n) = 1025n$

• $f(n) = 2n^2+11n-10$, $g(n) = 3n^2$

• $f(n) = n^2$, $g(n) = n^3$

• $f(n) = n^3$, $g(n) = n^2$

证明:

假设 $\exists$ 正常数 c 和 n_0 , 使得 $n \geq n_0$ 时, $n^3 \leq cn^2$ 成立。

构造 $n_1 = \max\{n_0, c\} + 1$, 显然有 $n_1 > n_0$ 且 $n_1 > c$, 显然 $n_1^3 > cn_1^2$, 与假设矛盾, 故 $n^3 \neq O(n^2)$ 。证毕。



O 性质

对于非负的 $f(n)$ 和 $g(n)$, 根据定义2.1, 有如下性质:

1. $O(f(n)) + O(g(n)) = O(\max(f(n), g(n)))$;

2. $O(f(n)) + O(g(n)) = O(f(n) + g(n))$;

3. $O(f(n)) O(g(n)) = O(f(n) g(n))$;

4. 如果 $g(n) = O(f(n))$, 则 $O(f(n)) + O(g(n)) = O(f(n))$;

5. $O(cf(n)) = O(f(n))$, 其中 c 是一个正的常数;

6. $f(n) = O(f(n))$ 。



证明 $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$

不妨设 $p(n) = \max\{f(n), g(n)\}$

设 $f'(n) = O(f(n))$, 则存在 c_1, n_1 , 当 $n \geq n_1$ 时, $f'(n) \leq c_1 * f(n)$

设 $g'(n) = O(g(n))$, 则存在 c_2, n_2 , 当 $n \geq n_2$ 时, $g'(n) \leq c_2 * g(n)$

则 $O(f(n)) + O(g(n)) = f'(n) + g'(n)$

当 $n \geq \max\{n_1, n_2\}$ 时,

$f'(n) + g'(n) \leq c_1 * f(n) + c_2 * g(n) \leq c_1 * p(n) + c_2 * p(n) = (c_1 + c_2) * p(n)$

即存在 $c_3 = c_1 + c_2$, $n_3 = \max\{n_1, n_2\}$, 当 $n \geq n_3$ 时, $f'(n) + g'(n) \leq c_3 * p(n)$

故 $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$



定理2.1

若 $A(n)=a_m n^m+\dots+a_1 n+a_0$ 是一个 m 次多项式, 则 $A(n)=O(n^m)$ 。

证明: 取 $n_0=1$, 当 $n \geq n_0$ 时

由 $A(n)$ 的定义和不等式关系 $|A+B| \leq |A|+|B|$ 有

$$\begin{aligned}|A(n)| &= |a_m n^m + \dots + a_1 n + a_0| \leq |a_m| n^m + \dots + |a_1| n + |a_0| \\&= (|a_m| + |a_{m-1}|/n + \dots + |a_0|/n^m) n^m \\&\leq (|a_m| + |a_{m-1}| + \dots + |a_0|) n^m\end{aligned}$$

取 $c = |a_m| + |a_{m-1}| + \dots + |a_0|$, 有 $|A(n)| \leq c n^m$

即: $A(n)=O(n^m)$, 定理得证。

定理2.1：若 $A(n)=a_m n^m + \dots + a_1 n + a_0$ 是一个 m 次多项式，则 $A(n)=O(n^m)$ 。

- 定理2.1表明，变量 n 的最高阶数为 m 的任一多项式，与 n^m 同阶。因此一个计算时间为 m 阶多项式的算法，其时间都可以用 $O(n^m)$ 来表示。
- 若一个算法有数量级为 $c_1 n^{m_1}$ ， $c_2 n^{m_2}$ ，... $c_k n^{m_k}$ 的 k 个语句，则算法的数量级及计算时间就是

$$c_1 n^{m_1} + c_2 n^{m_2} + \dots + c_k n^{m_k} = O(n^m), \quad m = \max\{m_i | 1 \leq i \leq k\}$$



数量级对算法有效性的影响

- 从计算时间上算法可以分为两类：
 - 多项式时间算法(**polynomial time algorithm**):用多项式来对其计算时间限界的算法
- 指数时间算法(**exponential time algorithm**):计算时间用指数函数限界的算法

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$$

$$O(2^n) < O(n!) < O(n^n)$$



不同时间复杂性函数的对比

T(n) 微秒	logn	n	nlogn	n ²	n ³	n ⁵	2 ⁿ	3 ⁿ	n!
n=10	3.3	10	33	100	1 毫秒	0.1 秒	1 毫秒	59 毫秒	3.6 秒
n=40	5.3	40	213	1600	64 毫秒	1.3 分	12.7 天	3855 世纪	10 ³ 世纪
n=60	5.9	60	354	3600	216 毫秒	7.6 分	366 世纪	1.3 × 10 ¹³ 世纪	10 ⁶⁶ 世纪

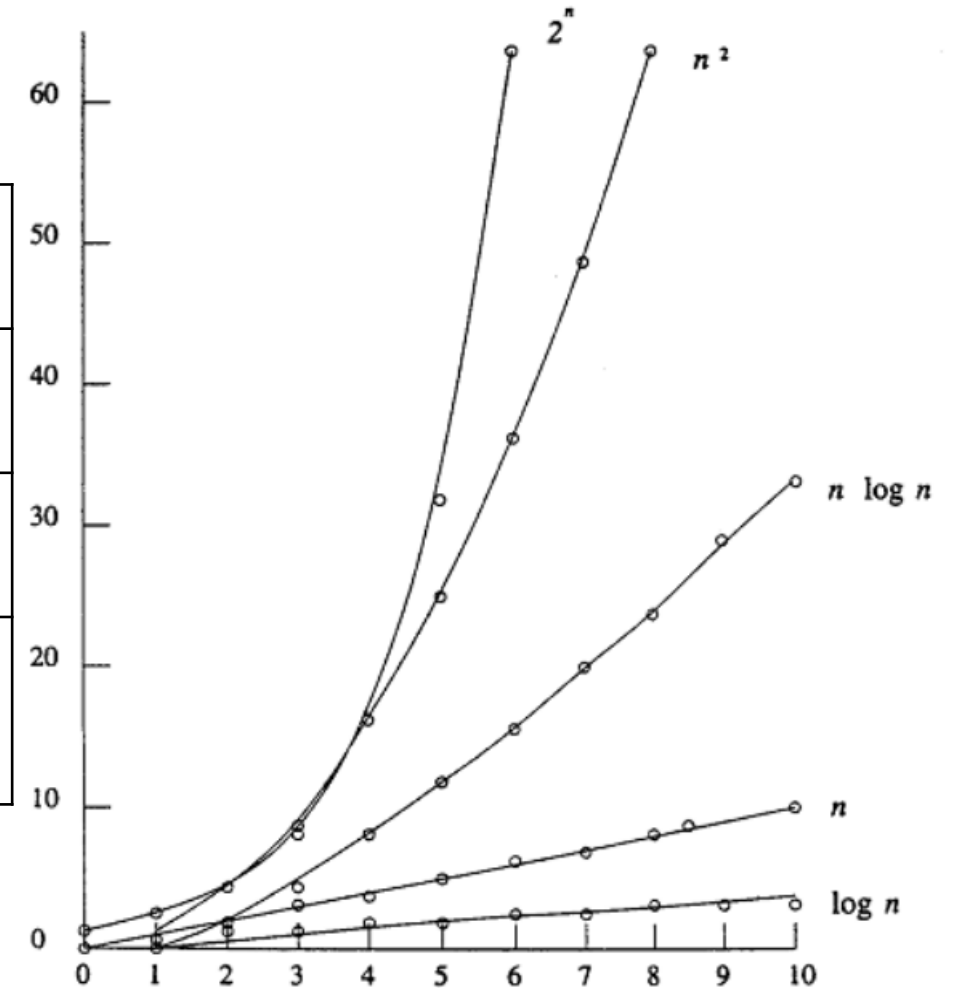


图2-1 时间复杂度函数曲线 blog.csdn.net/qq_41856733

对数函数的优越属性

- 对数函数的增长如此之慢，几乎可以认为：如果一个算法具有对数级的基本操作次数，该程序对任何实际规模的输入几乎都会在瞬间完成。

n	$\text{Log}_2 n$
10	3.3
10^2	6.6
10^3	10
10^4	13
10^5	17
10^6	20



定义2.2

如果存在两个正常数 c 和 n_0 ，对于所有的 $n \geq n_0$ ，有 $|f(n)| \geq c|g(n)|$ ，则记作： $f(n) = \Omega(g(n))$ 。

- 当 n 充分大时， $f(n)$ 有下界，一个常数倍的 $g(n)$ 是 $f(n)$ 的一个下界。
- $f(n)$ 的阶不低于 $g(n)$ 的阶。
- 在确定 $f(n)$ 的下界时，总是试图求出最大的 $g(n)$ 。
- 有关证明中，找出 c 和 n_0 是关键。



Ω 性质

对于非负的 $f(n)$ 和 $g(n)$, 根据定义2.2, 有如下性质:

1. $\Omega(f(n)) + \Omega(g(n)) = \Omega(\min(f(n), g(n)))$;
2. $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n) + g(n))$;
3. $\Omega(f(n)) \Omega(g(n)) = \Omega(f(n) g(n))$;
4. 如果 $g(n) = \Omega(f(n))$, 则 $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n))$;
5. $\Omega(cf(n)) = \Omega(f(n))$, 其中 c 是一个正的常数;
6. $f(n) = \Omega(f(n))$ 。



定义2.3

如果存在正常数 c_1, c_2 和 n_0 , 对于所有的 $n \geq n_0$, 有 $c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$, 则记作: $f(n) = \Theta(g(n))$ 。

- 一个算法的 $f(n) = \Theta(g(n))$ 意味着该算法在最好和最坏情况下的计算时间就一个常数因子范围内而言是相同的。



渐近表示函数的若干性质

- 传递性

- $f(n) = \Theta(g(n)), \quad g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n));$
- $f(n) = O(g(n)), \quad g(n) = O(h(n)) \Rightarrow f(n) = O(h(n));$
- $f(n) = \Omega(g(n)), \quad g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n));$

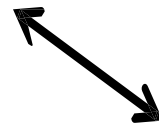
- 反身性

- $f(n) = \Theta(f(n));$
- $f(n) = O(f(n));$
- $f(n) = \Omega(f(n)).$



算法的时间复杂度

渐近记号	名称
Θ	渐近紧确界
O	渐近上界
Ω	渐近下界



输入情况	情况说明
最好情况	不常出现，不具普遍性
最坏情况	确定上界，更具一般性

算法运行时间称为算法的时间复杂度，通常使用渐近记号 O 表示



算法的时间复杂度

插入排序的时间复杂度

```
for  $j \leftarrow 2$  to  $n$  do  
   $key \leftarrow A[j]$   
   $i \leftarrow j - 1$   
  while  $i > 0$  and  $A[i] > key$  do  
     $A[i+1] \leftarrow A[i]$   
     $i \leftarrow i - 1$   
  end  
   $A[i+1] \leftarrow key$   
end
```

$O(n)$ $O(n^2)$

选择排序的时间复杂度

```
for  $i \leftarrow 1$  to  $n - 1$  do  
  for  $j \leftarrow i + 1$  to  $n$  do  
    if  $A[i] > A[j]$  then  
      交换  $A[i]$  和  $A[j]$   
    end  
  end  
end
```

$O(n)$ $O(n^2)$



常用的整数求和公式

$$\sum_{1 \leq i \leq n} 1 = \Theta(n)$$

$$\sum_{1 \leq i \leq n} i = n(n+1)/2 = \Theta(n^2)$$

$$\sum_{1 \leq i \leq n} i^2 = n(n+1)(2n+1)/6 = \Theta(n^3)$$

通式: $\sum_{1 \leq i \leq n} i^k = \frac{n^{k+1}}{k+1} + \frac{n^k}{2} + \text{低次项} = \Theta(n^{k+1})$



一些数学证明方法

- 直接证明: $P \rightarrow Q$
- 间接证明:
 - 反证法
 - 举反例
- 数学归纳法:
 - 初始归纳: $i=1$ 结论成立;
 - 归纳假设: 若 $i=n-1$ 时成立;
 - 归纳证明: 证明 $i=n$ 时成立。



作时空性能分布图

- 事后测试是在对算法进行设计、确认、事前分析和调试之后要做的工作，与所用计算机密切相关。
- 以时间分布图为例：
 - 为提高算法时间的准确测量，减少时钟不准确造成的噪声，可以增大规模或重复执行
 - 分布图的做法 -> 数据集与时间复杂度有关



小结

- 算法的设计——后续课程
- 算法的表示——伪代码
- 算法的确认——数学归纳法
- 算法的分析
 - 渐近上界 O
 - 渐近下界 Ω
 - 渐近紧确界 Θ





本章结束

